MorletDB: Recordings, Not Rows — Lag-Tolerant Waveform Retrieval in PostgreSQL

Version 0.11

Zin Nam

ABSTRACT

Analysts of waveform archives look for intervals that resemble a query up to a small time shift, and need the original samples of every hit. We treat this as top-k search under a bounded-lag cross-correlation scorer and ask how it can use a standard vector index inside PostgreSQL, and at what cost. On 235,000 seismic windows, a zero-lag shortlist of 1000 recovered only 74% of the scorer's top ten, whether taken exactly or from an HNSW index: the candidate metric, not approximate search, loses the matches. Because each lag term is an inner product with a zero-padded, shifted copy of the query, we send a sparse grid of 13 shifted probes to one unmodified inner-product HNSW index and re-rank their union with the unchanged scorer. On held-out seismic data, re-measured on one build, probe search recovered 97.7% of the exact top ten (lowest query 0.7) at a p95 latency of 57 ms, against 740 ms for the fastest exact path; zero-lag candidates from the same index recovered 0.639–0.658. Exact candidates on the sparse grid recovered 99.9%, and float16 probes added no observed loss. MorletDB returns hits as sample intervals of losslessly stored recordings. The costs are index structures of 0.97–1.34× the raw data, routing for selective filters, and write-ahead logging from index maintenance. All retrieval results are for seismic recordings.

1 INTRODUCTION

Condition monitoring, seismology and physiology produce archives of long, uniformly sampled recordings; condition monitoring of machines motivated this work. A common investigation starts from one interval and asks where similar behaviour occurred, then inspects the original samples and acquisition context of each candidate. Two practical requirements follow. Similarity must tolerate a small time shift, because a window grid rarely aligns two occurrences of the same event; and a result is useful only if it identifies an interval of a recording that can be read back exactly.

A bounded-lag cross-correlation captures the first requirement: two windows are similar if some shift of at most L samples correlates them strongly (Section 2). Exhaustive evaluation is linear in the archive. The obvious acceleration is a vector index, which PostgreSQL users already have through pgvector [1]; but such indexes rank by a fixed metric between unshifted vectors. This paper asks:

> *How can bounded-lag waveform search be accelerated with standard vector indexes while preserving the link to the recording, the semantics of filters and transactional updates, and which costs limit its use?*

Our argument has six steps. (1) Waveform investigations need intervals similar under a bounded shift, with access to their original samples. (2) Zero-lag vector search optimizes a different candidate ranking and loses neighbours of the defined scorer. (3) A diagnostic on the same data separates this effect from approximate search: an exact and an HNSW zero-lag shortlist lose the same neighbours (Section 6). (4) Shifted inner-product probes target exactly this difference: each lag term of the scorer is an inner product with a shifted query, so a few shifted queries against one index generate candidates for the scorer actually used (Section 3). (5) MorletDB places candidate generation, exact re-ranking and sample-interval results in PostgreSQL, next to losslessly stored recordings (Section 4). (6) Filters, index storage and maintenance cost determine the range in which this is worthwhile (Sections 8–10).

Contributions.

(1) **A specific candidate-generation method for a bounded-lag scorer:** query-side, zero-padded shifted probes on a sparse lag grid sent to an unmodified inner-product ANN index, a union of their candidates, and re-ranking with the unchanged scorer. Its ingredients are known (Section 11); the combination needs neither a trained embedding, nor per-shift copies of the data, nor lower bounds, at the price of approximate candidate selection (Section 3).
(2) **A recording-oriented integration in PostgreSQL:** results are sample intervals of losslessly stored recordings; search representations and their index rows are built by an explicit call that can share the append's transaction; eligibility filters and row-level security apply to both paths; a router chooses exact or probe search by the number of eligible windows (Section 4).
(3) **A controlled evaluation of benefit and limits:** a diagnosis of why zero-lag shortlists fail, an ablation that separates lag-grid sparsity from approximate search, confirmation on held-out stations and year, and measured costs of filters, storage and index maintenance (Sections 6–10).

Findings in brief. (1) *What works:* on held-out seismic data, shifted probes recovered 97.7% of the bounded-lag top ten at a p95 latency of 57 ms, against 740 ms for the fastest exact path (Table 2). (2) *Why:* zero-lag candidates rank by a different quantity and miss most matches whose best lag exceeds ±5 values, whether taken exactly or from HNSW (Figure 2); probes on a sparse lag grid cover them (Figure 3, Table 3). (3) *Cost:* a probe table and index of about one times the raw data, 13 index passes per query, slower search under filters that exclude a query's neighbours, and write-ahead logging from index maintenance (Sections 8–10). (4) *Not shown:* other signal domains, diagnostic relevance, cold caches, and operation beyond 111 minutes (Section 12).

Section 11 compares these with the closest prior work and states what we consider new. We do not claim a new similarity measure, a new index, lossless compression, or guarantees for probe search: probe search is approximate in the candidates it considers, and its recall is measured, not bounded. Recall in this paper is fidelity to the scorer, not relevance to a domain question, and all retrieval measurements use seismic recordings; condition monitoring motivated the work, but we show no diagnostic value for machines.

Code, frozen protocols and raw results are public [5]; a supplement reports storage, recovery and other studies not needed for the argument here.

2 PROBLEM AND SEARCH CONTRACT

A *recording* is a sequence $x_0, \ldots, x_{N-1}$ of float64 samples with a sample rate, a channel, a unit and JSON acquisition metadata. An *interval* $[a, b)$ denotes $x_a, \ldots, x_{b-1}$; sample offsets, not wall-clock times, are authoritative. A *window* is an interval of fixed length w on a stride grid. A search representation $\phi(x_{a:b}) \in \mathbb{R}^n$ is derived from a window by a versioned extractor.

We study envelopes: for $w = 4096$, z-score the window, take the magnitude of the analytic signal, smooth with a 50-sample moving mean, keep every tenth value and z-score again ($n = 410$). Stored envelopes are float16. For a query envelope q computed at full precision and a stored envelope g, the scorer is

$$D(q, g) \;=\; 1 - \max_{|\ell| \le L} \; \frac{1}{n} \sum_{t} q_{t+\ell}\, g_t, \qquad L = \lfloor 0.075\, n \rfloor = 30, \quad (1)$$

where terms with $t + \ell$ outside $[0, n)$ are zero. This is a *biased* cross-correlation: no per-lag renormalization of the overlapping part, no circular shift, and one z-score per window rather than per overlapping subsequence. At 100 Hz, $L = 30$ envelope values correspond to ±3 s.

Contract. A query names a source interval (or supplies samples), a metadata filter and k. The *eligible* windows are those whose recordings have the query's sample rate, channel and unit, whose window size and extractor match, whose metadata contain the filter, and which the caller may see under row-level security. The *exact result* is the first k eligible windows in the order $(D, \text{recording}, \text{start})$, after excluding windows overlapping the source interval and greedily skipping windows that overlap a better window of the same recording. Each result row is a recording identifier, an interval and its distance; the interval reads back the original samples bit-exactly from the raw store.

Three kinds of exactness. We keep apart (i) *exact distances*: every returned distance is D computed on the stored envelope; (ii) *exact candidate selection*: the returned set equals the exact result; and (iii) *relevance*: the returned intervals help answer the analyst's question. Probe search guarantees (i) but not (ii); we measure (ii) as recall@10 against the exhaustive search; neither implies (iii).

3 SHIFTED INNER-PRODUCT PROBES

For a lag ℓ, define the zero-padded shifted query $s_\ell[t] = q_{t+\ell}$ if $0 \le t + \ell < n$ and 0 otherwise. Then $\sum_t q_{t+\ell} g_t = \langle s_\ell, g \rangle$ and

$$1 - D(q, g) \;=\; \frac{1}{n} \max_{|\ell| \le L} \langle s_\ell, g \rangle. \quad (2)$$

The bounded-lag top-k is therefore the top-k of a maximum over $2L{+}1$ inner-product queries against the *unshifted* stored vectors. Two consequences matter. The shift is applied to the query, so the index holds one copy of each window, not one per lag. And the probes must be inner-product queries, not cosine: $\|s_\ell\|$ shrinks with $|\ell|$ because of zero padding, exactly as the biased scorer weights a shifted overlap.

Algorithm 1 replaces the full lag set by a sparse grid ($|\Lambda| = 13$ at spacing 5) and each exact inner-product top-c by an HNSW search.

Algorithm 1 Probe search (product function, simplified)

Require: query envelope q (full precision), eligibility predicate E, k; grid $\Lambda = \{-L, -L{+}5, \ldots, L\}$; per-probe budget $c = 50$; threshold $T = 5000$

```
if |{g : E(g)}| ≤ T then ▷ counted from recordings, stopping at T
    return ExactScan(q, E, k)
end if
C ← ∅
for ℓ ∈ Λ do
    C ← C ∪ top-c by ⟨f16(s_ℓ), g⟩ over g with E(g)    ▷ HNSW, iterative scan
end for
score every g ∈ C with D(q, g) on its stored envelope
return first k in order (D, rec, start) after source exclusion and overlap suppression
```

Three approximations are thus stacked, and the evaluation separates them: (a) *grid sparsity*: a window whose best lag lies between grid points must still rank in the top c of a neighbouring probe, which holds when envelopes are smooth relative to the spacing, but is not guaranteed; 13 probes at spacing five are an empirically chosen configuration for these envelopes, not a general setting; (b) *per-probe budget*: a window that is the global top ten under D need not be in any single probe's top c; (c) *index approximation*: HNSW returns an approximate top-c, and it compares a float16 copy of the probe with float16 stored vectors, whereas Equation 2 holds for the full-precision query. Re-ranking uses the full-precision query and the stored envelope, so (a)–(c) affect only which windows are considered, never a returned distance. The budget is fixed: probes are not repeated when source exclusion or overlap suppression leaves fewer than k windows, in which case the function returns fewer rows (the exact path instead fetches more rows until k survive); this never happened in our evaluation. Properties of D, such as tolerance to shifts within $\pm L$, do not transfer to the quantized index probes without measurement; Section 7 measures them.

4 INTEGRATION IN POSTGRESQL

MorletDB is a PostgreSQL 17 extension written in Rust with pgrx. Figure 1 shows the paths that matter for search; the supplement describes storage.

Recordings, not rows. The title names the user-facing abstraction, not a new kind of storage: domain sequence types have precedent, for example the temporal types of MobilityDB, which MEOS also provides as a library outside the database [36, 37]. Our recordings use integer sample offsets and half-open intervals rather than interpolated continuous time, and their coding is bit-exact. The scorer and the envelope exist once, in a Rust core used by the extension; evaluation code re-implements them in Python, checked against the core through fixtures.

Name. The system is named in honour of Jean Morlet and after its starting hypothesis: that Morlet-wavelet representations would suit search in technical signal archives. The implementation is not tied to the Morlet transform; representations are versioned, replaceable extractors, and the envelope used here is not a wavelet.

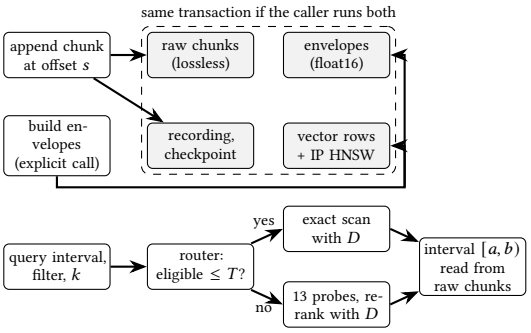


Figure 1: Write path and the two search paths. An append writes the raw chunk and advances the checkpoint; envelopes and, when probe search is enabled, their vector rows are written by an explicit builder call. All of them commit together when the caller runs append and builder in one transaction. Both search paths score candidates with D on stored envelopes and return sample intervals, which are read from the raw chunks, not from a representation. The exact path reads envelopes; the probe path reads vector rows for candidates and envelopes for re-ranking.

Our studies do not show a general advantage of Morlet wavelets over simpler representations (in sleep staging, FFT band powers did as well; supplement). The contribution of this paper is the retrieval method and its recording-oriented integration.

Recordings and appends. A collection is a set of caller-owned tables: recordings with a checkpoint, raw chunks, envelopes and, optionally, a pgvector table of `halfvec` rows with an inner-product HNSW index. An append names its global start offset and writes the raw chunk and the checkpoint in one transaction, so a committed checkpoint never refers to uncommitted samples. A repeated identical append is a verified no-op; a gap, overlap or changed retry fails. Raw chunks are coded losslessly (polynomial prediction, Rice coding and a float-to-integer grid with exact corrections; supplement), so a result interval reads back the appended float64 bit patterns. *An append does not build search representations.* Envelopes are written by an explicit call to `morlet_build_envelopes`, which also writes their vector rows if probe search was enabled for that window size when the call started. Samples and their representations become visible together only when the caller runs the append and the builder in the same transaction:

```
BEGIN;
SELECT morlet_append_chunk('signals', 'pump7', 81920, :x);
SELECT morlet_build_envelopes('signals', 'pump7');
COMMIT;   -- chunk, checkpoint, envelopes, vector rows
```

A builder call writes at most `max_windows` windows (10,000 by default) and returns how many it wrote; a call that hits this bound leaves later complete windows unbuilt until the next call, so a caller that needs every window covered repeats it until it returns 0. Windows of a representation may cross chunk boundaries, so a window becomes buildable only when the append that completes it has been made. Envelopes and vector rows are derived and can be dropped and rebuilt without touching raw data.

Two search paths, one contract. The exact path scores every eligible stored envelope with D in one parallel SQL statement. The probe path implements Algorithm 1; both apply the same eligibility, exclusion, ordering and suppression, and both return distances computed by the same Rust scorer. All functions run as `SECURITY INVOKER`, so privileges and row-level security apply to every table read. The probe path finds windows through the probe table and the exact path through the envelope table, so they search the same windows only when every stored envelope has its vector row. `morlet_build_envelopes` writes envelopes and, when probe search is enabled, their vector rows in one call and continues after the last stored window, which keeps each recording's searchable windows a gap-free prefix; envelopes written by other processes, or out of order, can leave windows without vector rows (reported as `probe_backlog` by `morlet_collection_status` and filled with one `INSERT` from the envelope table) or gaps that the status does not show. The exact functions run in the snapshot of the calling statement. The probe function issues several statements; under READ COMMITTED each takes its own snapshot, so a window appended or deleted during the call may be a candidate in one statement and absent from another, while under REPEATABLE READ or SERIALIZABLE all share one. In every case each returned row was visible, eligible and scored in the re-ranking statement's snapshot; we do not claim an atomic search over one snapshot under READ COMMITTED. Online index rebuilds (`REINDEX CONCURRENTLY`) keep the probe path usable while they run.

Filters. An approximate index answers "nearest overall"; a filter asks "nearest among eligible". Applying eligibility after probing silently discards candidates when few windows qualify. The router therefore counts eligible windows from the recordings table, stopping at the threshold T, and sends small galleries to the exact path; above T, each probe scans only eligible recordings, using pgvector's iterative index scan. Choosing between brute force and an index by filter selectivity is established practice in vector databases (Section 11); we include it because it decides whether probe search keeps the contract under filters.

Access control. Row-level security on the recordings table alone protects the search functions but not direct reads of the derived tables. `morlet_enable_row_security` therefore attaches a policy to every companion table that admits rows whose recording is visible; written as membership in an array of visible identifiers it keeps parallel scans (exact search +5%), whereas a correlated `EXISTS` tripled exact search time.

5 EVALUATION SETUP

Kinds of studies. *Confirmatory* studies committed their questions, data, arms, metrics and decision rules to the repository before their first result and ran once; deviations are logged in the protocols [5]. These are R1b, R2, BL1 and its confirmation BL1-C, BL1-S, SS1, SS3, E1 and the load runs; their protocols are internal plans, not registrations with a third party. *Exploratory diagnostics* had a protocol fixed before running but no confirmatory decision attached (R2-D, D1, D2) or none at all (the pruning feasibility study). *Post hoc* are the grid-spacing explanation and the choice of illustrative cases; the selection rule for the cases was committed before the figure was made, after the results were known. The

Table 1: Studies, data and measurement kind.

Study	Question	Data
R2, R2-D	exact scaling; zero-lag shortlists, exact vs HNSW	dev
BL1	probe grids and budgets, ablation controls	dev
BL1-C	frozen configuration, once	held-out
SS3	filtered search and router; product check	dev, held-out
SS1, R1b	one stored copy; float16 precision	dev, held-out; 2022
E1	**all arms on one build**	**dev, held-out**
D1, D2	time per stage; filter correlation	dev
BL1-S, C2–C4	index maintenance during ingestion	dev; dev+held-out

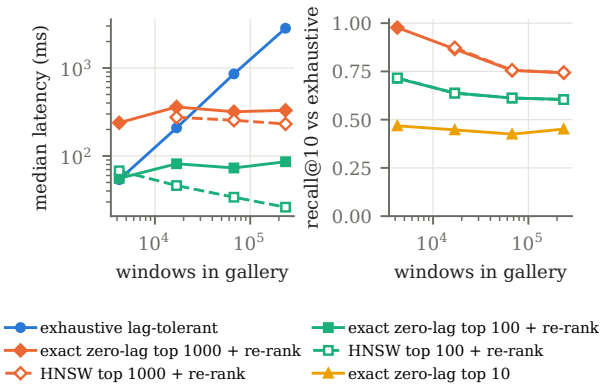


Figure 2: R2, R2-D on dev galleries. Left: median latency of the exhaustive scan (historic, single-threaded implementation) and of zero-lag shortlists plus re-ranking (exact: solid; HNSW: dashed). Right: recall@10; the HNSW curve lies on the exact one. 100 queries per gallery.

supplement lists each study with its commits. Table 1 lists the studies used here.

Data. Development data (*dev*) are CX HHZ recordings [4] of stations PB01, PB02, PB06 and PSGCX, 1–14 March 2025: 481 M samples at 100 Hz, 234,762 windows of 4096 samples at stride 2048. Held-out data (*held-out*) are stations PB05, PB11, PB14 and PB18, 1–14 March 2021 (234,622 windows), chosen from the data centre's availability listing before any waveform was downloaded; no earlier study had read them. Each set has 100 queries from its first day: 50 random and 50 high-energy windows. Seismic envelopes are one signal domain; we discuss generality in Section 12.

Consolidated comparison. The main retrieval tables come from E1, a comparison of all arms on one build. E1 re-measured the held-out data that BL1-C had already used for its single confirmation (and SS3 reused), with configurations frozen before BL1-C; it is therefore a consolidated re-measurement, not another independent confirmation. It ran on `main` at commit 68594d7, with one configuration, the same queries and one session, under a protocol frozen before the run [31]. PostgreSQL 17.11 ran with default settings (128 MB shared buffers, 4 MB `work_mem`, two parallel workers per query; one exact arm allowed eight, of which the planner used three; PostgreSQL 17 has no asynchronous I/O), pgvector 0.8.0 (HNSW m=16, `ef_construction` 64), warm caches, one client, on an Apple M5 Max (18 cores, 128 GB) with no other CPU-intensive job (load average 3.8–4.7, mostly E1 itself). Each arm ran twice per query; latencies use all 200 timings. In both databases every envelope had its vector row, every recording's windows formed a gap-free prefix, and nothing was written during the runs, so all arms searched the same complete set of windows. Later fixes to the extension (row-security set-up, a race between routing and concurrent commits, a race between enabling probes and concurrent builds) change none of the statements measured here [31].

Earlier studies. The studies that explain design decisions (R2-D, BL1, SS1, SS3) used earlier implementation states and a machine shared with an unrelated CPU-intensive job; we label their numbers as such. Recall@10 is always against the exhaustive exact result.

6 WHY ZERO-LAG SHORTLISTS FAIL

A natural first stage takes the top K windows by zero-lag cosine from an ANN index and re-ranks them with D. R2 measured this on dev galleries nested from 4,217 to 234,762 windows [7]. R2's statement passed its query vector through a subquery, which pgvector cannot use as an index ordering key, so it silently computed an *exact* zero-lag top K. A diagnostic on the same data (R2-D, protocol frozen before measuring) therefore compared an explicitly exact shortlist with an HNSW shortlist whose plan was checked to use the index.

With K=1000, recall@10 fell from 0.977 on the smallest gallery to 0.743 on the largest; with K=100, from 0.714 to 0.605 (Figure 2). The HNSW shortlist agreed with the exact one on 98–100% of candidates on the largest gallery and gave the same recall to within 0.007 at every size (0.744 at K=1000). Every top-ten item contained in a shortlist survived re-ranking and suppression, so coverage equalled recall. The loss is therefore caused by the zero-lag candidate metric, not by approximate search or by the contract's exclusion rules: a quarter of the bounded-lag top ten lies outside the 1000 windows nearest by zero-lag cosine. The exact zero-lag top ten alone shared 43–47% of the bounded-lag top ten. These are two budgets and one HNSW configuration. E1 reproduced this on the current build: a zero-lag HNSW shortlist of 1000 from the probe index recovered 0.744 on dev and 0.658 on held-out data. The exhaustive scan in Figure 2 is the historic implementation (linear, 12.1–12.8 µs per window); the current exact path runs as one parallel statement and is the baseline below.

7 SHIFTED PROBES: ABLATION AND CONFIRMATION

BL1 chose the configuration on dev data: grids of 7 and 13 probes and budgets of 50 and 100 per probe all met its gate (recall@10 $\geq$ 0.95, p95 at most a third of the exhaustive scan and below the fastest exact path, added storage at most 1.5× raw); 13 probes ×

Table 2: E1, held-out data (234,622 windows, 100 queries, 200 timings per arm). Recall@10: mean / minimum / queries below 0.9.

Arm	Recall@10	p50 / p95 ms	Work per query
exact, 2 workers	1	958 / 971	score all windows
exact, 8 allowed (3 used)	1	728 / 740	score all windows
probes (product)	**0.977 / 0.7 / 5**	39 / **57**	13 scans, re-rank ≈350
zero-lag HNSW, 650	0.639 / 0.1 / 69	28 / 40	1 scan, re-rank 650
zero-lag HNSW, 1000	0.658 / 0.1 / 67	34 / 50	1 scan, re-rank 1000

Table 3: E1: recall@10 by candidate generator. All arms re-rank with D.

Candidates	dev	held-out
all 61 lags, exact (PX-61)	1.000	1.000
13-lag grid, exact (PX-13)	1.000	0.999
13-lag grid, float16 probes, exact (PQ-13)	1.000	0.999
7-lag grid, exact (PX-7)	0.986	0.987
13-lag grid, HNSW (product)	**0.998**	**0.977**
zero-lag, exact, same count (EZ)	0.681	0.613

50 was frozen and confirmed once on held-out data [10]. E1 then measured the product functions and all controls on one build [31]. Its offline controls use the same stored float16 envelopes and the same re-ranking: *PX-61*, the exact top 50 for each of the 61 lags; *PX-13* and *PX-7*, the same for the 13- and 7-lag grids; *PQ-13*, as PX-13 with each probe rounded to float16, as the index sees it; and *EZ*, an exact zero-lag shortlist with as many candidates as PX-13.

Benefit. On held-out data the product probe function recovered 97.7% of the exact top ten at a p95 latency of 57 ms against 740 ms for the fastest exact path, a ratio of 13 between the p95 latencies (Table 2); on dev data, 99.8% at 53 against 783 ms. Zero-lag candidates from the same index recovered 64–66%, even with a larger re-rank budget. Equal candidate counts are not equal work: the 650-row zero-lag arm traverses the index once with a wide search list, the probe arm thirteen times with a narrow one. No arm returned fewer than ten rows, and the two rounds on the same index returned identical lists; a rebuilt HNSW index can return different candidates. By query group, held-out recall was 0.998 for the 50 random windows and 0.956 for the 50 high-energy windows; energy is a selection rule for queries, not a measure of relevance. Recall is the share of the exact ten that a result contains, averaged over queries; every exact reference had ten rows. Of the 100 held-out queries, 83/12/4/1 reached 1.0/0.9/0.8/0.7; the 23 missed intervals (of 1,000) ranged from rank 1 to 10 in the exact list, 3 were the exact best match, and none was replaced by an overlapping window of the same recording. A recall of 0.7 thus means that three of the ten exact intervals are absent and lower-ranked admissible intervals take their place. The factor 13 is the ratio of the two arms' p95 latencies over 200 timings each; probe p95 was 58.5 and 49.7 ms in the two rounds, exact p95 740 and 738 ms. It is not a per-query speed-up. These numbers supersede the experiment-harness figures reported earlier for BL1-C (131 ms against 713 ms), which came from an earlier implementation on a shared machine.

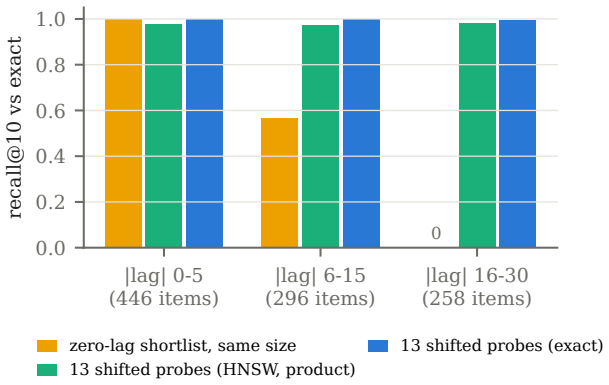


Figure 3: E1, held-out: recall of the exact top ten by the lag at which each item scores best.

Where the loss comes from. Table 3 separates the approximations of Section 3. Each control keeps the 50-row budget per probe and the stored float16 envelopes. On held-out data, exact candidate generation on the sparse grid recovered 0.999 of reference intervals (99.9%); rounding the probes to float16 introduced no further observed loss (0.999). HNSW candidate generation reduced recall to 0.977. Most observed loss therefore arose from approximate candidate selection, with one reference interval already missed by the sparse grid. On dev data the grid lost nothing and HNSW 0.002. A grid at spacing ten lost 0.013–0.014. HNSW with full-precision probes cannot be measured separately, because the `halfvec` index takes float16 queries. Missing probe rows, differences between searched windows and concurrent writes are separate possible sources of loss; they were excluded here by construction and by the checks above. None of this guarantees the same split on other data or with other HNSW settings. The six hardest held-out queries are all high-energy windows of one station, and each reaches 1.0 with exact grid candidates. Figure 3 shows the mechanism: zero-lag candidates find items whose best lag is within ±5 but almost none beyond ±15, whereas probes are flat across lags. Figure 4 shows one case of each kind.

Why spacing five sufficed here. For each of the 1,000 reference items per database, the best of the 13 grid probes kept a median 0.999 of the item's best-lag correlation (5th percentile 0.98, minimum 0.84), and ranked the item at median 2 and 95th percentile 7–8 among all windows in that probe's exact ordering; one held-out item ranked 324th, outside the 50-row budget, which is the single PX-13 miss. On 3,000 dev envelopes the envelope's own biased autocorrelation is 0.973 at an offset of two values and 0.895 at five [39], consistent with the smoothing applied before decimation (about five envelope values). This explains the observed behaviour of spacing five on these data; it guarantees neither top-k preservation nor recall on other signals, because what decides recall is the rank inside a probe, and closely spaced scores can reorder even when most of the correlation is kept.

Where the time goes. An instrumented copy of the probe function on dev data (D1) [31] spent 65% of its time in the 13 index statements (1.9 ms each, p50) and the rest in re-ranking, reading the

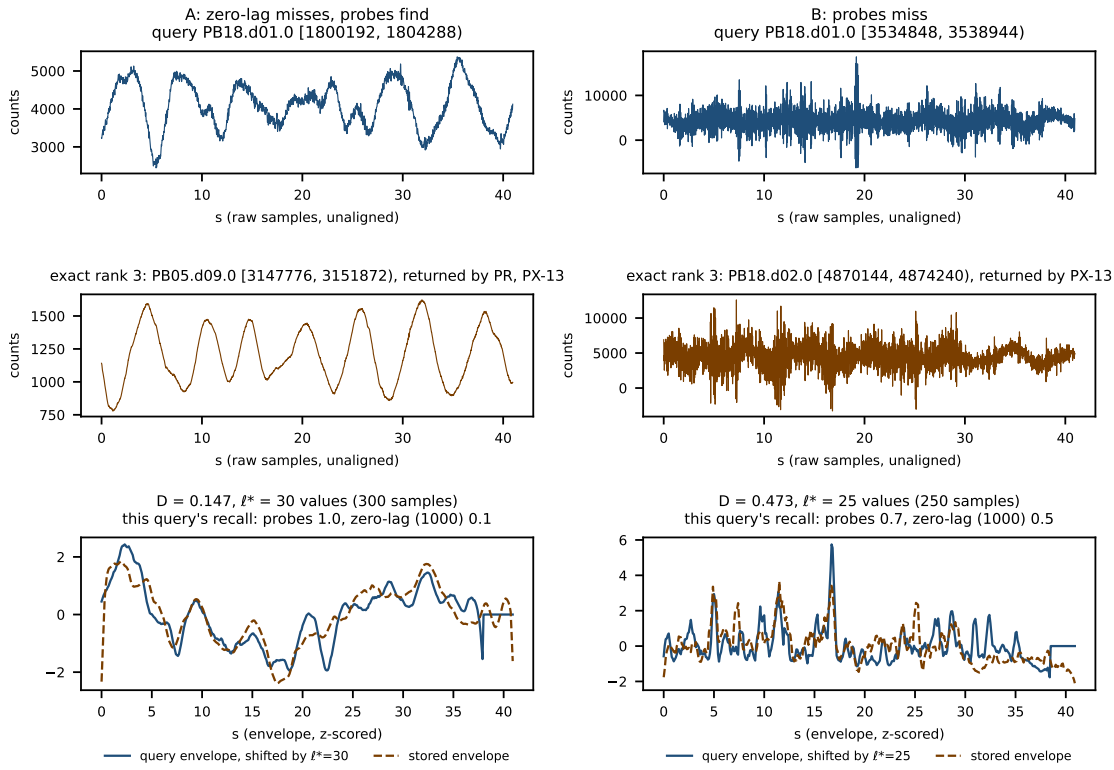


Figure 4: Two held-out cases, selected by a rule fixed before the figure was made: (A) the query with the largest recall gain of probes over a zero-lag shortlist of 1000, and the reference interval with the largest best lag among those the probes find and the shortlist misses; (B) the query with the lowest probe recall, and its best-ranked missed interval, which exact grid candidates (PX-13) do find. They illustrate, and do not replace, the distribution in Table 2; no domain label is implied. Top rows: raw samples in counts, unaligned. Bottom: z-scored envelopes as the scorer uses them, the query shifted by the best lag ℓ^* with zero padding; the distance D is the product scorer's and was checked against $1 - \max_\ell c(\ell)/n$.

source interval and routing; instrumentation added about 13 ms to the product's 29 ms, mostly by materializing the re-rank, so the index statements account for most of the product's time. The eligibility predicate cost 0.03 ms per statement. Issued on its own, one index statement took 0.38 ms to plan and 0.82 ms to execute (`EXPLAIN ANALYZE`); inside the function each probe statement took about 1.9 ms. Planning inside the function was not measured, so the remaining 0.7 ms per probe is unresolved; planning of the dynamically issued statement, building the probe vector and collecting the result are candidates. ANN traversal, heap access and visibility checks happen inside one index-scan node and were not separated. Every probe returned 50 rows; the union held 335 windows (median), one was excluded as the source, none was suppressed.

Table 4: SS3 (earlier build, shared machine): filtered search on held-out data, mean recall@10 (product: also minimum) against exact search, 100 queries per level. Post-f.: post-filtered probes; F-aware: filter-aware probes; Routed: product function (*routed to exact).

Eligible	Exact p95	Post-f.	F-aware	Routed mean/min	Routed p95
1.8% (4,217)	68 ms	0.515	0.998	**1.0**/1.0*	85 ms
7.2% (16,868)	329 ms	0.848	0.997	**0.997**/0.9	207 ms
25% (58.6 k)	290 ms	0.972	0.986	**0.986**/0.6	98 ms
29% (67.5 k)	390 ms	0.976	0.994	**0.994**/0.8	88 ms
100% (234.6 k)	~1.2 s	0.976	0.977	**0.977**/0.7	69 ms

8 FILTERS AND ROUTING

Investigations usually restrict the gallery to one sensor or period. The first probe implementation applied eligibility after probing. SS3 compared three strategies at five filter levels per query [12]: exact search, post-filtered probes and filter-aware probes; a router threshold was fixed on dev data before the held-out run.

Post-filtering lost up to half the top ten (Table 4). Filter-aware probes restored recall but were slow at the smallest level (p95

724 ms), where the planner abandoned the index and exact search took 68 ms. The router sends that level to exact search. No returned row violated its filter, and with row-level security restricting a role to one station, every strategy returned only that station and rejected a hidden recording as query source. The router missed its pre-registered latency bound once, at the unfiltered held-out level, by 0.8 ms. Low per-query minima at 25% and 100% coincide with the high-energy queries of Section 7, not with routing errors.

Correlated filters. A filter can keep the same number of windows but more or fewer of a query's neighbours, which changes how far an index scan must travel to find eligible rows [34]. D2 chose, per dev query, the day filter (about 7% of windows) or station filter (25%) that held the most, a random, or the fewest of the query's unfiltered top hundred; the choice is synthetic [31]. Recall was 0.997–1.0 in every case. Latency was not: with day filters, probe p95 rose from 121 ms (positive) to 178 (neutral) and 227 ms (negative), against 210 ms for exact search on the now quiet machine; with station filters, probes stayed at 58–63 ms against 264 ms. Routing by the number of eligible windows therefore preserved recall in this workload, but its latency crossover depends on how the filter relates to the query, and the threshold, set on a shared machine, sends 7%-filters to probes where exact search is as fast.

9 STORAGE, RECALL AND LATENCY

Probe search costs space. In E1, raw chunks took 371 MB (dev) and 514 MB (held-out), envelopes 223 MB (read by both paths for scoring), the probe table 223 MB and its HNSW index 275 MB: the probe structures added 1.34× and 0.97× the raw data. Rebuilding the index took 39–41 s with 4 GB of maintenance memory. Smaller storage and faster search are separate goals: float16 probes did not change recall (PQ-13), and storing fewer bytes need not make search faster. How large envelopes are relative to raw data depends on stride and on how well the raw data compress (supplement).

Float16 storage is not the bottleneck: on 2022 windows that no earlier study had read, float16 envelopes kept recall@10 at 0.9999 against envelopes recomputed from raw samples [6]. The two stored copies hold the same float16 values in 98.3% of positions. SS1 tested keeping one [11]: a single `halfvec` table was exact on both databases (identical distances for 9.4 M pairs) and saved about 250 MB per 235,000 windows, but made exact scans 23% slower; an HNSW index on an expression over the envelope table kept exact search unchanged but raised probe p95 by 27–32%. Both missed their pre-registered latency bounds. The trade-off is therefore three-way: about 1–1.3× raw storage bought a p95 latency 13 times lower than the fastest exact path in E1, at recall 0.977; saving about half of that storage cost 20–30% latency on one path in SS1.

10 MAINTENANCE DURING INGESTION

Maintaining vector rows at commit makes new windows searchable immediately, but every append now updates an HNSW graph. Two measurements bound this cost; neither is a long-term stability result.

Replay (BL1-S, dev, one day of four channels, 16,868 windows). With the builder called in each append transaction, every window was searchable at its commit while appends ran at 919,000 samples/s, 53% of raw-only throughput. At 200× real time, probe queries during ingestion had a p95 of 82 ms against 189 ms for exact search, at recall 0.9997 on the same snapshot [10].

Sustained load (15-minute runs). On an archive of 938,768 windows, four writers appended 16 channels at 25.6 kHz (409,600 samples/s), calling the builder in each append transaction, while four clients searched [13]. With default settings the run met its criteria (probe p95 337 ms, spot-check recall 0.998, ingest lag at most 22 s) but wrote 13.8 GB of write-ahead log for 0.76 GB of growth; at twice the rate the writers fell behind. With four settings changed together (8 GB shared buffers, 16 GB `max_wal_size`, 15-minute checkpoints, WAL compression) the same run wrote 1.91 GB for 0.65 GB, with probe p95 100 ms; because the settings changed together and background load differed (11.7 against 7.3), neither the WAL reduction nor the latency difference is attributed to a single cause. Details are in the supplement.

Longer run (C4). A run planned for two hours with the four settings above met the target for 111 minutes: 409,585 samples/s, writer lag at most 24 s in the first hour (with the unrelated job running) and 3.4 s in the second, probe p95 138 ms at recall 0.993, no errors, and 12.0 GB of WAL for 4.7 GB of growth [13]. Then every client process paused for 90–106 s, the writers fell 91 s behind, and the harness's lag rule, stricter than the protocol's wording, stopped the run at 112.5 minutes. The cause is unknown; the server log shows no error, which does not rule out a system-side cause. The planned two-hour run therefore did not complete, and the criteria were not changed afterwards. C4 concerns operation under ingest; it does not bear on the retrieval comparison, which ran on static databases. These results describe a bounded pilot workload on one machine; they are not a production qualification.

11 RELATED WORK AND WHAT IS NEW

We looked for the strongest overlapping prior work, reading primary sources; Table 5 records for each work whether we read the full text (F), parts (P) or only the abstract (A). The full matrix, with quotations and what we searched for, is in the repository [5]. We use *shift* for a displacement along the time axis. Invariance to a vertical offset (adding a constant), as in normalization-based methods [29], and shift-invariant kernels that average over all circular shifts [19] are different notions.

What is known. Three ideas we use are established. (i) *Shifting the query instead of the data.* Rafiei [14] evaluates similarity under a set of transformations, explicitly including time shifts, against one index of untransformed sequences, and processes all transformations in one index traversal; Keogh et al. [15] expand a query into all its rotations, keep the database stored once, support rotation-limited queries by dropping rows of that matrix, and merge neighbouring rotations into a few envelopes, an exact counterpart of our sparse grid. Foulds et al. [17] search under cyclic permutations of the query. All three are exact, use Euclidean-type distances, and treat shifts as circular. (ii) *Several vector queries against one approximate index, union, exact re-ranking.* ColBERT's end-to-end retrieval issues one ANN search per query vector, takes the union of document identifiers and re-ranks it exhaustively [24]. (iii) *ANN candidates for cross-correlation.* In seismology, Tibi et

Table 5: Closest prior work. Shift: which displacements along the time axis the similarity tolerates. Side: where the shift is applied. Cands.: whether the candidate set is exact for the stated similarity. Read: F full text, P parts, A abstract. [a]exact for the magnitude feature, which itself approximates the rotation distance.

Work	Similarity	Shift	Side	Index	Cands.	Copies, refinement	Read
Rafiei 1999 [14]	Euclidean on normal form, truncated DFT	set of transformations incl. time shifts (circular in DFT)	transformations applied to index entries	R-tree on DFT coefficients, one traversal for all	exact	one copy; false positives removed exactly	F
Keogh et al. 2006/09 [15]	Euclidean (also DTW)	all circular rotations or a limited range	query: matrix of rotations, merged into wedges	lower bounds; VP-tree	exact	one copy; exact refinement	F
Vlachos et al. 2005 [16]	Euclidean on Fourier magnitudes	all circular	data: invariant feature	VP-tree variant	exact[a]	one compressed signature	P
Foulds et al. 2015 [17]	distance over cyclic permutations of the query	all cyclic	query	none; parallel DFT-based search	exact	—	A
k-Shape, TODS 2017 [8, 18]	normalized CC incl. biased variant	all linear shifts (FFT)	pairwise	none	exact	—	P
GRAIL 2019 [19]	kernel over all circular shifts of NCC	all circular	data: learned embedding	any (Euclidean/IP)	approx.	one embedding per series	P
Qiu et al. 2018 [20]	Pearson at zero lag	none	data: learned embedding	Euclidean index	approx.	one embedding	P
MASS, UCR suite [3, 9]	z-normalized ED per subsequence	every offset	scan	none	exact	—	A
FAST, Rong et al. [2, 21]	Jaccard on binary fingerprints	data windows every 1 s	data: dense windows	MinHash LSH	approx.	one fingerprint per overlapping window	F
Tibi et al. 2017 [22, 23]	max-over-lag CC of waveforms	bounded lags	data: kernel-PCA embedding of CC	KD-tree forest	approx.	projected vectors; exact CC on candidates	F
ColBERT 2020 [24]	sum of max inner products	—	query: one vector per token	one ANN index, one search per vector	approx.	union of candidates re-ranked exactly	F
This paper	biased, zero-padded CC	linear, $\|\ell\| \le L$	query: sparse grid of shifted copies	unmodified IP HNSW, one search per probe	approx.	one vector copy + index; union re-ranked with D	—

al. [22] project waveforms with kernel PCA so that correlation maps to Euclidean distance, search a KD-tree forest and evaluate exact correlation on the returned set (25× faster than linear search at recall above 80%); Gonzales et al. [23] issue several internal queries to compensate for projection error. FAST obtains shift tolerance from densely overlapping data windows [2, 21], and GRAIL learns embeddings of a shift-invariant correlation kernel [19]. The identity between a lag term and an inner product with a shifted signal is textbook; the biased statistic itself appears as NCC_b in the k-Shape work [18]. On the database side, choosing brute force over an index when a filter is selective is standard in vector systems (AnalyticDB-V, Milvus, ACORN, and the documented thresholds of Weaviate, Qdrant, Vespa and Lucene) [25–27]; VBase and pgvector's iterative scans provide filtered index scans inside PostgreSQL [1, 32]; SingleStore-V makes vector search visible at commit by indexing immutable segments and scanning fresh rows exactly [28]; and KV-match and ULISSE return exact subsequence matches as offsets in stored series [29, 30].

Vector search inside PostgreSQL. VBASE exposes an index as an iterator that the executor can resume until enough rows pass the filters, and stops when the traversal moves away from the query [32]; its guarantee is equivalence to a top-K-then-filter plan with the best K, which needs filters that commute with sorting. Our score is a maximum over lags of which only the grid is probed, and greedy overlap suppression does not commute with sorting, so that argument does not carry over; pgvector's iterative scan already resumes within each probe statement, and E1 and D1 found no query whose fixed budget ran out. PostgreSQL-V moves index contents out of PostgreSQL pages and indexes recent rows asynchronously, giving up the WAL-logged, commit-time index visibility that our contract uses [33]. A study of filtered vector search in a PostgreSQL-compatible engine shows that correlation between filter and query neighbourhood, not only selectivity, governs the cost of iterative scans [34]; D2 observed the same effect on latency, not on recall. MorletDB uses no PostgreSQL hooks and no copied internals, the patterns that a survey of extensibility associates with incompatibilities; its dependencies are PostgreSQL 17 and pgvector 0.8 or later [35].

What is new here. We claim neither query-side shifting, nor multi-query candidate generation, nor routing as new. What we did not find in the works above, which is not evidence that it does not exist, is their combination for a *bounded, zero-padded, linear* lag scorer on an *unmodified, approximate inner-product index*: Equation 2 lets a bounded-lag score use any inner-product ANN index without training an embedding (unlike Tibi et al. and GRAIL), without replicating windows per shift (unlike FAST), and without lower bounds (unlike Rafiei and Keogh et al.). The price is that candidate selection becomes approximate, which those exact methods avoid. The contribution we can support with evidence is therefore narrower than a new algorithm: (1) a measured diagnosis that zero-lag candidates fail for this scorer because of their metric, not because of approximate search; (2) an ablation separating grid sparsity from index error, with an explanation of the grid spacing from envelope smoothness; (3) a held-out confirmation; and (4) an integration that keeps one contract on both paths, with measured costs of storage, filters and maintenance. Implementing the method in PostgreSQL

is not by itself a contribution; keeping intervals, filters and row-level security identical on the exact and approximate paths, and making built windows visible at the commit that builds them, is the part we evaluate.

The single-traversal alternative. Rafiei and Keogh et al. show that processing all transformations in one exact traversal with lower bounds can beat one search per transformation. Before building probe search we examined the two lag-uniform bounds closest to this design on our scorer [38]: a Keogh-style envelope bound (one wedge over all lags $|\ell| \le L$) and a spectral bound, on 1,826 windows of 2022 data. This was a feasibility study, not an implemented search: even under idealized assumptions (the true tenth-best distance as threshold, bounds computed at no cost) their combination pruned 63% of candidates, because it overestimated the best correlation by a median factor of 1.83. Inserted into the measured per-candidate costs of the exact scan of that time, this gives a modelled speed-up of about 1.9× on that gallery and 2.3× in the limit of large galleries. These figures are idealized model values for this construction and cost model; they are not measurements, they are not comparable one-to-one with the measured probe speed-up in Section 7, and they do not bound exact methods in general: finer wedges (one per group of neighbouring lags), other bounds or a faster exact scorer could do better. They explain why we pursued approximate candidates rather than exact pruning.

12 LIMITATIONS

One signal domain. All retrieval results are for seismic envelopes from one network, and our empirical claims are limited to these data. The method needs only a bounded-lag inner-product scorer, but how many probes and candidates suffice depends on the representation's smoothness and on where best lags fall relative to the grid; we have not measured this on vibration, acoustic or physiological data. Condition monitoring motivated the work and is a target application; we have no evidence that probe search, or envelope similarity in general, is diagnostically relevant for machine condition.

Earlier implementation states. E1 measured the main comparison on one build. The filtered-search table (SS3), the single-copy study (SS1), the zero-lag scaling study (R2, R2-D) and the load runs were measured on earlier states and on a machine shared with another job; ratios across those studies mix states.

Measurement conditions. One laptop-class machine, warm caches, one HNSW configuration. The load tests ran 15 and 111 minutes; the planned two-hour run stopped after an unexplained pause, and we make no claim about longer operation, archives beyond about one million windows, cold caches or several tenants under load.

What recall means. Recall@10 measures fidelity to the scorer D. Whether the bounded-lag top ten are the intervals an analyst wants is a separate question; we have a catalogue-distance proxy for seismic events in the supplement, but no task evaluation of probe search.

Approximation without bounds. Probe search can miss a window that the exact path returns (per query, down to 7 of 10 on held-out data); the budget is fixed and is not extended when exclusion or suppression removes candidates. The idealized lower-bound study of Section 11 covered one construction; finer bounds were not tested.

Routing. The router's threshold counts eligible windows only. D2 showed that its latency crossover depends on how the filter relates to the query and on machine load; we did not recalibrate it.

Grid choice. The 13-probe grid was chosen on dev data and explained post hoc by envelope smoothness; it was confirmed on held-out data, for this representation only.

Kinds of evidence. *Demonstrated* on these data: zero-lag shortlists lose a quarter of the top ten for this scorer, exactly or with HNSW; exact grid candidates recover 0.999 on held-out data, a measurable loss of one interval in 1,000; float16 probes add no further observed loss; HNSW lowers recall to 0.977; random queries reach 0.998 and high-energy queries 0.956; the measured costs of storage, filters and maintenance. *Empirical explanation:* spacing five suffices because envelopes are smooth relative to the spacing (Section 7); index statements dominate probe time (D1). *Inconclusive:* the 0.7 ms per probe statement that D1 did not resolve; which of four jointly changed PostgreSQL settings reduced write-ahead logging; whether the system runs stably for two hours, since C4 stopped at 112.5 minutes. *Open hypotheses:* that the grid and budget transfer to other smooth representations and signals; that scorer fidelity translates into relevance for any domain task.

When to use it. In the conditions we tested, probe search paid off for unfiltered or weakly filtered galleries of about a quarter of a million windows, where it was 4–15 times faster at p95 than exact search at recall 0.977–1.0 (Tables 2 and 4, D2). Exact search was the better or equal choice for small galleries (a few thousand eligible windows) and for selective filters that exclude the query's neighbourhood, and it is the only choice when every true neighbour must be found. Galleries of other sizes, other signals, other hardware and cold caches are untested; the crossover points measured here are not general thresholds.

13 CONCLUSION

Bounded-lag waveform search can use a standard inner-product index if the index is asked the right question. A zero-lag shortlist asks a different one, and lost a quarter of the scorer's top ten whether it was computed exactly or approximately. Because each lag term is an inner product with a shifted query, thirteen shifted probes into one unmodified HNSW index, re-ranked with the unchanged scorer, recovered 97.7% of the top ten on held-out seismic stations and year with a p95 latency 13 times lower than that of the fastest exact path, in a comparison of all arms on one build; exact candidates on the sparse grid recovered 99.9%, float16 probes added no further observed loss, and approximate candidate selection accounted for most of the loss. Inside PostgreSQL, one contract holds on both paths: hits are sample intervals of losslessly stored recordings, filters and row-level security apply, and windows become searchable at the commit of the transaction that builds them. The benefit is bounded by three costs: probe structures of 1.0–1.3× the raw data; a router that must send selective filters to exact search, with a latency crossover that depends on how filters relate to the query; and write-ahead logging from index maintenance, which was 18 times the database growth with default settings and 2.9

times at a second operating point that differed in four settings and in background load. A second signal domain is the evidence still missing for a claim beyond seismic data.

USE OF AI ASSISTANCE AND RESPONSIBILITY

Code, experiment harnesses, analyses and text of this report were developed with Claude (Anthropic), an AI assistant, working under the author's direction. Where a study had a protocol, it was committed before measurement (Section 5 and the supplement); the author reviewed the results and is responsible for the content and for any errors.

REFERENCES

[1] pgvector contributors. pgvector: open-source vector similarity search for Postgres, version 0.8.0. https://github.com/pgvector/pgvector.

[2] Clara E. Yoon, Ossian O'Reilly, Karianne J. Bergen, and Gregory C. Beroza. 2015. Earthquake detection through computationally efficient similarity search. *Science Advances* 1, 11, e1501057. doi:10.1126/sciadv.1501057.

[3] Thanawin Rakthanmanon, Bilson Campana, Abdullah Mueen, Gustavo Batista, Brandon Westover, Qiang Zhu, Jesin Zakaria, and Eamonn Keogh. 2012. Searching and Mining Trillions of Time Series Subsequences under Dynamic Time Warping. In *Proc. 18th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD '12)*, 262–270. doi:10.1145/2339530.2339576.

[4] GFZ German Research Centre for Geosciences and Institut des Sciences de l'Univers–Centre National de la Recherche CNRS-INSU. 2006. IPOC Seismic Network. Integrated Plate boundary Observatory Chile. Network code CX. doi:10.14470/PK615318.

[5] MorletDB contributors. 2026. MorletDB source repository, protocols and results. https://github.com/merkhet-io/morletdb.

[6] MorletDB contributors. R1b confirmation (2022). `evaluation/materialized/compact/results/conf-2022/RESULTS.md` in [5].

[7] MorletDB contributors. R2: retrieval scaling, and R2-D diagnostic. `evaluation/retrieval_scaling/` in [5].

[8] John Paparrizos and Luis Gravano. 2015. k-Shape: Efficient and Accurate Clustering of Time Series. In *Proc. 2015 ACM SIGMOD Int. Conf. on Management of Data (SIGMOD '15)*. doi:10.1145/2723372.2737793.

[9] Abdullah Mueen et al. The Fastest Similarity Search Algorithm for Time Series Subsequences under Euclidean Distance. https://www.cs.unm.edu/~mueen/FastestSimilaritySearch.html.

[10] MorletDB contributors. BL1: shifted-probe retrieval, confirmation and ingestion. `evaluation/bounded_lag/` in [5].

[11] MorletDB contributors. SS1: one stored envelope representation. `evaluation/search_storage/SS1_RESULTS.md` in [5].

[12] MorletDB contributors. SS3: filtered envelope search. `evaluation/search_storage/SS3_RESULTS.md` in [5].

[13] MorletDB contributors. Pilot stages 1–2: access control, recovery and load. `evaluation/pilot/RESULTS.md` in [5].

[14] Davood Rafiei. 1999. On Similarity-Based Queries for Time Series Data. In *Proc. 15th Int. Conf. on Data Engineering (ICDE)*, 410–417. doi:10.1109/ICDE.1999.754957.

[15] Eamonn Keogh, Li Wei, Xiaopeng Xi, Sang-Hee Lee, and Michail Vlachos. 2006. LB_Keogh Supports Exact Indexing of Shapes under Rotation Invariance with Arbitrary Representations and Distance Measures. In *Proc. 32nd Int. Conf. on Very Large Data Bases (VLDB)*, 882–893. Extended as: Eamonn Keogh, Li Wei, Xiaopeng Xi, Michail Vlachos, Sang-Hee Lee, and Pavlos Protopapas. 2009. Supporting exact indexing of arbitrarily rotated shapes and periodic time series under Euclidean and warping distance measures. *The VLDB Journal* 18, 3, 611–630. doi:10.1007/s00778-008-0111-4.

[16] Michail Vlachos, Zografoula Vagena, Philip S. Yu, and Vassilis Athitsos. 2005. Rotation Invariant Indexing of Shapes and Line Drawings. In *Proc. 14th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 131–138. doi:10.1145/1099554.1099580.

[17] Leslie R. Foulds, Jorge P. de Morais Neto, Humberto J. Longo, Hugo A. D. do Nascimento, and Wellington S. Martins. 2015. A variant of k-nearest neighbors search with cyclically permuted query points for rotation-invariant image processing. *Discrete Applied Mathematics* 197, 123–144. doi:10.1016/j.dam.2015.02.011.

[18] John Paparrizos and Luis Gravano. 2017. Fast and Accurate Time-Series Clustering. *ACM Trans. Database Syst.* 42, 2, 1–49. doi:10.1145/3044711.

[19] John Paparrizos and Michael J. Franklin. 2019. GRAIL: Efficient Time-Series Representation Learning. *Proc. VLDB Endow.* 12, 11, 1762–1777. doi:10.14778/3342263.3342648.

[20] Han Qiu, Hoang Thanh Lam, Francesco Fusco, and Mathieu Sinn. 2018. Learning Correlation Space for Time Series. arXiv:1802.03628.

[21] Kexin Rong, Clara E. Yoon, Karianne J. Bergen, Hashem Elezabi, Peter Bailis, Philip Levis, and Gregory C. Beroza. 2018. Locality-Sensitive Hashing for Earthquake Detection: A Case Study of Scaling Data-Driven Science. *Proc. VLDB Endow.* 11, 11, 1674–1687. doi:10.14778/3236187.3236214.

[22] Rigobert Tibi, Christopher Young, Antonio Gonzales, Sanford Ballard, and Andre Encarnacao. 2017. Rapid and Robust Cross-Correlation-Based Seismic Signal Identification Using an Approximate Nearest Neighbor Method. *Bull. Seismol. Soc. Am.* 107, 4, 1954–1968. doi:10.1785/0120170011. Read as the OSTI accepted manuscript (OSTI 1372350).

[23] Antonio Gonzales, Nicholas Blazier, et al. Seismic Waveform Matching via Approximate Nearest Neighbor with Projection Deficiency Compensation. Sandia National Laboratories, OSTI 1458306.

[24] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proc. 43rd Int. ACM SIGIR Conf.*, 39–48. doi:10.1145/3397271.3401075.

[25] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data. *Proc. VLDB Endow.* 13, 12, 3152–3165. doi:10.14778/3415478.3415541.

[26] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, et al. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proc. 2021 Int. Conf. on Management of Data (SIGMOD)*, 2614–2627. doi:10.1145/3448016.3457550.

[27] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proc. ACM Manag. Data* 2, 3, 1–27. doi:10.1145/3654923.

[28] Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, Chun Wu, et al. 2024. SingleStore-V: An Integrated Vector Database System in SingleStore. *Proc. VLDB Endow.* 17, 12, 3772–3785. doi:10.14778/3685800.3685805.

[29] Jiaye Wu, Peng Wang, Ningting Pan, Chen Wang, Wei Wang, and Jianmin Wang. 2019. KV-Match: A Subsequence Matching Approach Supporting Normalization and Time Warping. In *Proc. 35th IEEE Int. Conf. on Data Engineering (ICDE)*, 866–877. doi:10.1109/ICDE.2019.00082.

[30] Michele Linardi and Themis Palpanas. 2018. Scalable, Variable-Length Similarity Search in Data Series: The ULISSE Approach. *Proc. VLDB Endow.* 11, 13, 2236–2248. doi:10.14778/3275366.3275372.

[31] MorletDB contributors. E1: consolidated retrieval comparison, and diagnostics D1–D2. `evaluation/consolidated/RESULTS.md` in [5].

[32] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao

Yang, and Lidong Zhou. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *17th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 377–395.

[33] Jiayi Liu, Yunan Zhang, Chenzhe Jin, Aditya Gupta, Shige Liu, and Jianguo Wang. 2026. Fast Vector Search in PostgreSQL: A Decoupled Approach. In *Conf. on Innovative Data Systems Research (CIDR)*.

[34] Duo Lu, Helena Caminal, Manos Chatzakis, Yannis Papakonstantinou, Yannis Chronis, Vaibhav Jain, and Fatma Özcan. 2026. An In-Depth Study of Filter-Agnostic Vector Search on a PostgreSQL Database System. *Proc. ACM Manag. Data* 4, 3, Article 134. doi:10.1145/3802011.

[35] Abigale Kim, Marco Slot, David G. Andersen, and Andrew Pavlo. 2025. Anarchy in the Database: A Survey and Evaluation of Database Management System Extensibility. *Proc. VLDB Endow.* 18, 6, 1962–1976. doi:10.14778/3725688.3725719.

[36] Esteban Zimányi, Mariana Duarte, and Víctor Diví. 2024. MEOS: An Open Source Library for Mobility Data Management. In *Proc. 27th Int. Conf. on Extending Database Technology (EDBT)*, demonstration, 810–813. doi:10.48786/edbt.2024.76.

[37] Mahmoud Sakr, Esteban Zimányi, Alejandro Vaisman, and Mohamed Bakli. 2023. User-centered road network traffic analysis with MobilityDB. *Transactions in GIS* 27, 2, 323–346. doi:10.1111/tgis.12972.

[38] MorletDB contributors. Certified pruning for envelope search: headroom and feasibility. `evaluation/pruning/README.md` in [5].

[39] MorletDB contributors. Grid-spacing analysis on development data. `research/paper/analysis/` in [5].

A CLAIMS AND EVIDENCE

Each claim with the study that supports it, the measurement kind (E: offline experiment control, P: product functions) and data (D: dev, C: held-out), and its main restriction. Protocols, results and commands: [5]; E1 and diagnostics: [31].

Claim	Study	Restriction
zero-lag top 1000 + re-rank: 0.743 (exact), 0.744 (HNSW)	R2-D; E, D	earlier build; two budgets
HNSW and exact shortlists share 98–100% of candidates	R2-D; E, D	largest gallery
zero-lag HNSW 1000 from probe index: 0.744 / 0.658	E1; P, D / C	one HNSW configuration
probes 0.977 (min 0.7; 5/100 <0.9), p95 57 ms	E1; P, C	seismic; unfiltered; quiet machine
fastest exact p95 740 ms (8 workers allowed)	E1; P, C	PG 17 defaults, warm cache
probes 0.998, p95 53 vs 783 ms	E1; P, D	development data
exact grid candidates 0.999 (1 of 1,000 missed); float16 probes no further loss	E1; E, C	offline controls
HNSW candidates: 0.999 → 0.977 (held-out)	E1; P vs E	this configuration only
random 0.998, high-energy 0.956 (held-out)	E1; P, C	energy is a query rule, not relevance
83/12/4/1 held-out queries at 1.0/0.9/0.8/0.7	E1; P, C	3 of 23 misses were rank 1
same complete universe on both paths	E1 check	static databases
7-lag grid loses 0.013–0.014	E1; E, D+C	offline control
grid probe keeps ≥0.98 correlation (p5), rank ≤8 (p95)	E1; E, D+C	explains, does not guarantee
no short results; identical lists on the same index	E1; P, D+C	rebuilt index may differ

Claim	Study	Restriction
index statements ≈65% of probe time; 0.7 ms per probe unresolved	D1; P, D	instrumented copy
correlated filters: recall 0.997–1.0, p95 121–227 ms	D2; P, D	synthetic filter choice
post-filter 0.515 at 1.8% eligible; routed 1.0	SS3; E/P, C	earlier build
routed recall 0.977–1.0, no filter or RLS violations	SS3; P, D+C	earlier build
probe structures 0.97–1.34× raw; rebuild 39–41 s	E1; P	depends on raw compressibility
one stored copy: −250 MB, +23% or +27–32% p95	SS1; E, D+C	earlier build
lag-uniform bounds prune ≤63% (idealized)	pruning; E	one construction; model
searchable at commit; 919k samples/s	BL1-S; E, D	one day, four channels
409.6k samples/s: 15 min (defaults), 111 min (tuned)	C2, C4; P	2-h run not completed
WAL/growth 18× (defaults), 2.9× (second point)	C2, C2p; P	four settings and load differed
companion-table RLS: exact +5%	Stage 1; P	one tenant layout